

Matplotlib

Matplotlib is a powerful plotting library in Python used to create static, interactive, and animated visualizations. It is especially good for line charts, bar charts, scatter plots, and more.

1. Installation

To install matplotlib:

```
pip install matplotlib
```

2. Basic Structure

Every matplotlib visualization follows a basic flow:

1. Import matplotlib.pyplot
2. Create data
3. Call plotting functions
4. Customize the plot
5. Display the plot using plt.show()

3. First Simple Plot

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4]
y = [10, 20, 25, 30]
plt.plot(x, y)
plt.show()
```

Explanation:

- `plt.plot(x, y)` plots a line graph connecting the points (1,10), (2,20), etc.
- `plt.show()` renders the plot.

4. Plotting with Titles and Labels

```
plt.plot(x, y)
plt.title("Sales Over Days")
plt.xlabel("Days")
plt.ylabel("Sales")
plt.show()
```

Explanation:

- `plt.title()` sets the title of the plot.
- `plt.xlabel()` sets the x-axis label.
- `plt.ylabel()` sets the y-axis label.

5. Customizing Line Style

```
plt.plot(x, y, color='green', linestyle='--', marker='o')
plt.title("Dashed Line with Markers")
plt.xlabel("Days")
plt.ylabel("Sales")
plt.show()
```

Parameters:

- `color='green'`: Line color
- `linestyle='--'`: Dashed line
- `marker='o'`: Circle marker at each point

6. Multiple Lines on Same Plot

```
x = [1, 2, 3, 4]
y1 = [10, 20, 25, 30]
y2 = [5, 15, 20, 28]

plt.plot(x, y1, label='Product A', color='blue')
plt.plot(x, y2, label='Product B', color='red')
plt.title("Product Sales Comparison")
plt.xlabel("Days")
plt.ylabel("Sales")
plt.legend()
plt.show()
```

Explanation:

- `plt.legend()` displays the legend using the `label=` values.

7. Saving a Plot

```
plt.plot(x, y)
plt.title("Simple Plot")
plt.savefig("plot_image.png")
plt.show()
```

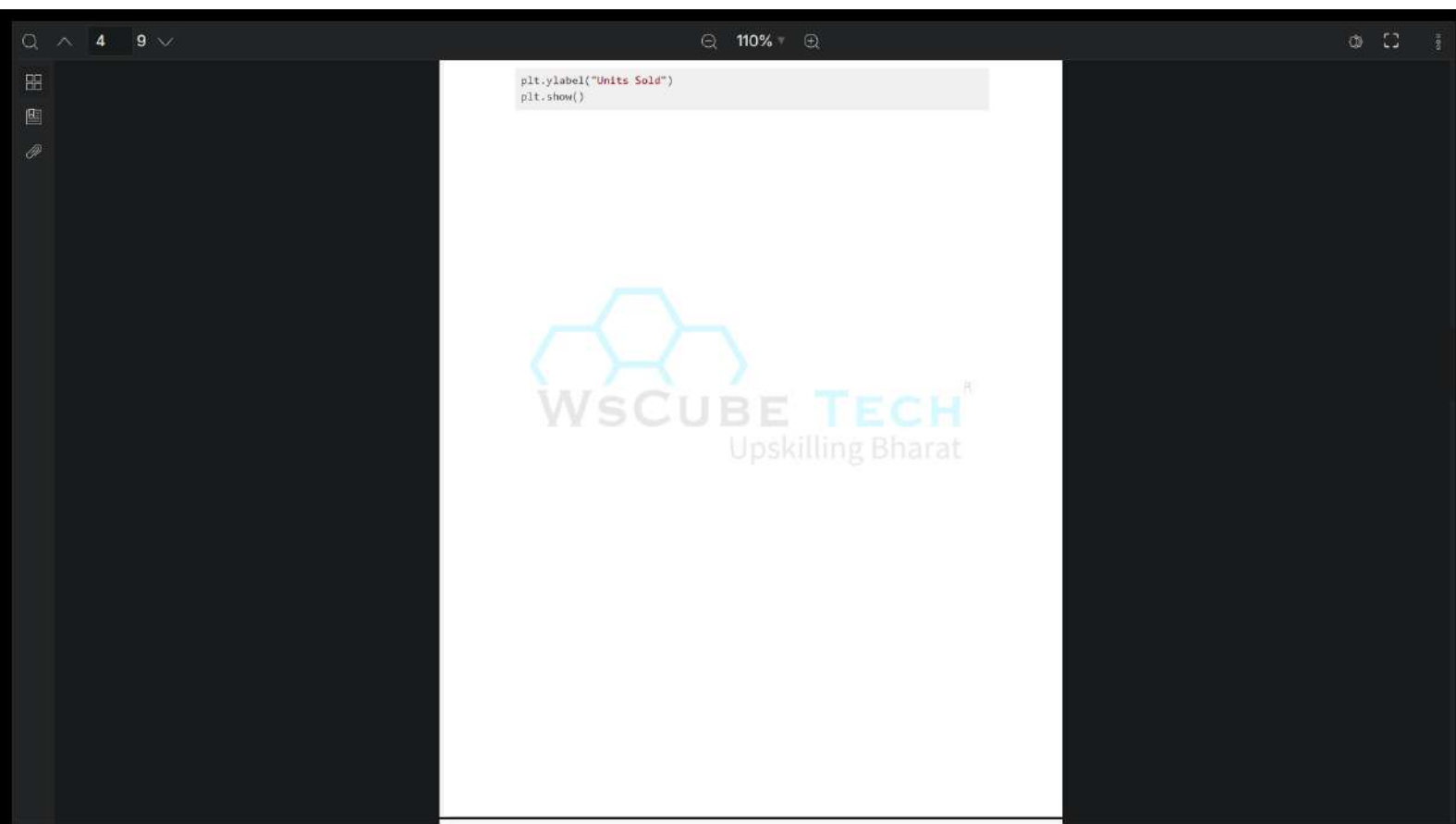
Explanation:

- `plt.savefig()` saves the figure to an image file before showing it.

8. Bar Chart

```
products = ['Pen', 'Notebook', 'Eraser']
sales = [100, 80, 50]

plt.bar(products, sales)
plt.title("Product Sales")
plt.xlabel("Product")
```



9. Scatter Plot

```
x = [5, 7, 8, 7, 2, 17, 2, 9]
y = [99, 86, 87, 88, 100, 86, 103, 87]

plt.scatter(x, y)
plt.title("Study Hours vs Scores")
plt.xlabel("Hours Studied")
plt.ylabel("Score")
plt.show()
```

10. Pie Chart

```
labels = ['Pen', 'Notebook', 'Eraser']
sizes = [40, 35, 25]

plt.pie(sizes, labels=labels, autopct='%1.1f%%')
plt.title("Product Share")
plt.show()
```

Matplotlib - Advanced

1. Figure Size and DPI (Resolution)

You can set the figure size and resolution using `plt.figure()`.

```
plt.figure(figsize=(8, 4), dpi=100) # width=8 inches, height=4 inches,
resolution=100 dpi
x = [1, 2, 3, 4]
y = [10, 20, 30, 40]
plt.plot(x, y)
plt.title("Custom Size Plot")
plt.show()
```

2. Subplots (Multiple Plots in One Figure)

Use `plt.subplot()` or `plt.subplots()` to display multiple charts side by side or in a grid.

Method 1: `plt.subplot()`

```
x = [1, 2, 3, 4]
y1 = [1, 4, 9, 16]
y2 = [10, 20, 25, 30]

plt.subplot(1, 2, 1) # (rows, columns, position)
plt.plot(x, y1)
plt.title("Square Numbers")

plt.subplot(1, 2, 2)
plt.plot(x, y2)
plt.title("Linear Growth")

plt.tight_layout() # Avoids overlapping
plt.show()
```

Method 2: `plt.subplots()`

```
fig, axes = plt.subplots(2, 1) # 2 rows, 1 column
axes[0].plot(x, y1)
axes[0].set_title("Square")

axes[1].plot(x, y2)
axes[1].set_title("Linear")

plt.tight_layout()
plt.show()
```

3. Grid Lines

```
plt.plot(x, y1)
plt.grid(True) # Enable grid
plt.title("Grid Enabled Plot")
plt.show()
```

You can customize the grid:

```
plt.grid(color='gray', linestyle='--', linewidth=0.5)
```

4. Axis Range (Zoom In/Out)

```
plt.plot(x, y1)
plt.xlim(0, 5) # Set X-axis range
plt.ylim(0, 20) # Set Y-axis range
plt.title("Custom Axis Range")
plt.show()
```

5. Annotating Specific Points

Use `plt.annotate()` to label specific points.

```
plt.plot(x, y2)
plt.annotate("Peak", xy=(4, 30), xytext=(3, 32),
            arrowprops=dict(facecolor="red", shrink=0.05))
plt.title("Annotated Plot")
plt.show()
```

6. Styling with Matplotlib Stylesheets

Matplotlib provides built-in themes.

```
plt.style.use('ggplot') # Try 'seaborn', 'fivethirtyeight', 'bmh', etc.

plt.plot(x, y2)
plt.title("Styled Plot")
plt.show()
```

To reset to default:

```
plt.style.use('default')
```

7. Rotating X-axis Labels

```
products = ['Pen', 'Notebook', 'Eraser', 'Pencil', 'Sharpener']
sales = [18, 20, 15, 18, 12]

plt.bar(products, sales)
plt.xticks(rotation=45)
plt.title("Rotated X Labels")
plt.show()
```

8. Custom Tick Marks and Labels

```
x = [1, 2, 3, 4, 5]
y = [100, 200, 300, 400, 500]

plt.plot(x, y)
plt.xticks([1, 2, 3, 4, 5], ['Mon', 'Tue', 'Wed', 'Thu', 'Fri'])
plt.title("Custom Tick Labels")
plt.show()
```

9. Adding Text Inside Plot

```
plt.plot(x, y)
plt.text(3, 300, "Midweek peak!", fontsize=12, color='blue')
plt.title("Text in Plot")
plt.show()
```

10. Adding Horizontal and Vertical Lines

```
plt.plot(x, y)
plt.axhline(y=300, color='r', linestyle='--') # Horizontal line at y=300
plt.axvline(x=3, color='g', linestyle=':') # Vertical line at x=3
plt.title("Threshold Lines")
plt.show()
```

4. Real-World Mini Project Ideas Using Matplotlib

